Reviewer Pack — Free Probability Invariant Bounds Communication Complexity o…

Entry 7173b29a4549 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:35:06 UTC

Free Probability Invariant Bounds Communication Complexity of Disjointness

Entry ID: `7173b29a4549` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 09:35:06 UTC

1. Conjecture

Field A (mathematical branch): free probability **Field B** (complexity object): communication complexity

Statement:

> ['For every n-ary boolean function f, let C(f) be its characteristic polynomial. Define the free entropy H_f as the von Neumann entropy of the positive semi-definite matrix whose entries are the non-negative coefficients of C(f). Then, for the disjointness problem on n bits, there exists a constant c > 0 such that the randomized communication complexity CC_DISJ_n is lower-bounded by c * log(H_F)', 'where F denotes the characteristic polynomial of the adjacency matrix of an n-vertex graph randomly chosen from the set of all n-vertex graphs.', 'This conjecture holds for n ≤ 40.']

Rationale (proposer's reasoning):

> ['Free probability has been used to study the entanglement of quantum states, but its application to communication complexity is novel. The characteristic polynomial and free entropy provide a quantifiable measure that could potentially expose non-trivial structures in the disjointness problem.', 'The proposed invariant connects the algebraic properties of boolean functions with the geometric nature of graphs, suggesting a possible avenue for proving lower bounds on CC_DISJ_n.']

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ee303623d7d01680`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the computed H_F values correlate with a communication complexity CC_DISJ_n ≥ c * log(H_F) for some constant c > 0, and no seed produces a CC_DISJ_n < c * log(H_F). The conjecture is falsified if any seed yields a CC_DISJ_n < c * log(H_F) or support_fraction < 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free probability" AND "communication complexity" AND disjointness" - "characteristic polynomial" free entropy communication complexity" - "von Neumann entropy" "positive semi-definite matrix" lower-bounds CC_DISJ_n"

Top relevant hits considered: - [s2:1196bb270e8514d054c22d46934d95d4f32a59ca] The optimality of Dantzig estimator for estimating low rank density matrices ∗ - [s2:1610.04811] Estimation of low rank density matrices by Pauli measurements

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        return [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    def characteristic_polynomial(matrix):
        n = len(matrix)
        if n == 0:
            return [1]
        elif n == 1:
            return [matrix[0][0] - 1, 1]

        det = 0
        for j in range(n):
            sub_matrix = [[matrix[i][k] for k in range(n) if k != j] for i in range(1, n)]
            det += ((-1) ** j) * matrix[0][j] * characteristic_polynomial(sub_matrix)
        return [det]

    def free_entropy(matrix):
        n = len(matrix)
        coeffs = characteristic_polynomial(matrix)
        total = sum(coeffs)
```

```
        entropy = 0
        for coeff in coeffs:
            if coeff > 0:
                p = coeff / total
                entropy -= p * math.log2(p)
        return entropy

    def communication_complexity(n):
        # Placeholder function; replace with actual computation
        return random.random() * n

    n_values = [5, 10, 15, 20, 30, 40]
    instances_tested = 0
    total_cc_disj_n = 0.0
    total_h_f = 0.0

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            graph = generate_random_graph(n)
            h_f = free_entropy(graph)
            cc_disj_n = communication_complexity(n)

            if h_f > 0:
                total_cc_disj_n += cc_disj_n
                total_h_f += h_f
                instances_tested += 1

    mean_cc_disj_n = total_cc_disj_n / instances_tested
    mean_h_f = total_h_f / instances_tested

    if instances_tested < 30:
        return {
            "metric_name": "communication_complexity",
            "metric_value": None,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    c = mean_cc_disj_n / math.log2(mean_h_f)
    conjecture_holds = all(cc_disj_n >= c * math.log2(h_f) for h_f, cc_disj_n in zip(total_h_f, to

    return {
        "metric_name": "communication_complexity",
        "metric_value": mean_cc_disj_n,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"c={c}, {mean_cc_disj_n} < {c * math.log2(m
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"c={results[first_failing_seed]['counterexample'
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de522cb7.py", line 97, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de522cb7.py", line 60, in run_trial
    h_f = free_entropy(graph)
          ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de522cb7.py", line 39, in free_entropy
    coeffs = characteristic_polynomial(matrix)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de522cb7.py", line 34, in characteristi
    det += ((-1) ** j) * matrix[0][j] * characteristic_polynomial(sub_matrix)
                                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de522cb7.py", line 34, in characteristi
    det += ((-1) ** j) * matrix[0][j] * characteristic_polynomial(sub_matrix)
                                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_de522cb7.py", line 34, in characteristi
    det += ((-1) ** j) * matrix[0][j] * characteristic_polynomial(sub_matrix)
                                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 1 more time]
TypeError: unsupported operand type(s) for +=: 'int' and 'list'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed during execution, which prevents us from verifying the conjecture's support or falsification. | next: Investigate and fix the error in the characteristic_polynomial function to ensure the test can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11803
2	preregistration	ollama_remote	glm4:latest	0	0	6066
3	novelty	ollama_remote	glm4:latest	0	0	4641
4	novelty	ollama_remote	glm4:latest	0	0	5635
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15356
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10467
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11406
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17967
9	judge	ollama_remote	glm4:latest	0	0	13335

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 96675 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7173b29a4549.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7173b29a4549.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7173b29a4549.tar.gz` (if generated)